

Low-Cost AI Accelerator

Design Specification

MSR-4 Weight Encoding and Compensation

Current scope

Introduction, Background, and Motivation

The architecture, control, verification, and PPA chapters will be added as the project progresses.

Contents

1	Introduction	2
2	Background	3
2.1	INT8 Quantization for DNN Inference	3
2.2	Most Significant Runs	3
2.3	MSR-4 Weight Encoding	4
3	Motivation	6
3.1	Why MSR-4 Is the Selected Operating Point	6
3.2	MSR-4 Exception Density and Compensation Capacity	6
3.3	Expected-Value Approximation and Sparse Compensation	7
3.4	Verification Implications	7

Introduction

This section is reserved for the project problem statement, contributions, and document organization. It will be completed after the proposed architecture and evaluation scope are finalized.

Background

2.1 INT8 Quantization for DNN Inference

Deep neural networks are normally trained with floating-point arithmetic, but the cost of floating-point weights and MAC operations is often unnecessary during inference. A signed INT8 representation reduces weight storage, memory traffic, and multiplier complexity while retaining a practical accuracy baseline. Let w be a floating-point weight and s be the quantization scale. The signed integer weight used by the accelerator is

$$q = \text{clip}_{[-128,127]}(\text{round}(sw)), \quad (2.1)$$

where q is stored as an 8-bit two's-complement word. For an activation vector x and a quantized weight vector q , the INT8 reference operation is

$$y = \sum_{i=0}^{K-1} x_i q_i. \quad (2.2)$$

INT8 treats every weight with the same storage width and multiplier interface, even though the bit patterns of trained weights are not uniformly distributed. In particular, weights close to zero contain redundant sign-extension bits after quantization. The proposed design exploits this representation-level redundancy while retaining INT8 as the external quantization interface and the accuracy reference.

2.2 Most Significant Runs

For an N -bit two's-complement word, an MSR- k pattern is a run of k identical bits at the most significant end. This specification uses $N = 8$ and selects $k = 4$. Let

$$q = (b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0)_2, \quad (2.3)$$

where b_7 is the sign bit. The MSR-4 predicate is

$$\mathcal{M}_4(q) = 1 \iff b_7 = b_6 = b_5 = b_4. \quad (2.4)$$

Thus, both $0000xxx_2$ and $1111xxx_2$ are MSR-4 values. Figure 2.1 illustrates why the pattern is common after fixed-point quantization: small positive values generate leading zeros, while small negative values generate leading ones.

0.004096	-0.420437	0.153534	-0.197340		<u>00000001</u>	<u>11001010</u>	<u>00010100</u>	<u>11100111</u>
0.114851	0.036513	-0.028751	0.189524		<u>11110001</u>	<u>00000101</u>	<u>11111100</u>	<u>00011000</u>
0.174329	-0.028819	0.143463	0.159553	→	<u>00010110</u>	<u>11111100</u>	<u>00010010</u>	<u>00010100</u>
0.243178	0.036531	-0.082088	0.139659		<u>11100001</u>	<u>00000101</u>	<u>11110101</u>	<u>00010010</u>
0.025325	-0.313233	0.173153	0.173304		<u>00000011</u>	<u>11010111</u>	<u>00010110</u>	<u>11101010</u>
Original Data					Fixed Point			

Figure 2.1: Example conversion from floating-point weight data to fixed-point words. Underlined leading bits illustrate MSR candidates.

For example, with scale $s = 128$, 0.10534 maps to $13 = (00001101)_2$ and has four leading zeros. Similarly, -0.0784 maps to $-10 = (11110110)_2$ and has four leading ones. In both cases, the four most significant bits independently carry no additional information once the remaining field and the class are known.

2.3 MSR-4 Weight Encoding

The Weight Pre-processing Unit (WPU) classifies every INT8 weight using (2.4). It emits one 5-bit reduced word r for the main array and a compensation-valid bit v_c . The corresponding encoding is defined in Table 2.1. The leading bit of r is a class tag: zero identifies an MSR-4 reduced word and one identifies a non-MSR-4 high-nibble contribution.

Table 2.1: MSR-4-Aware Per-Weight Encoding

Class	Reduced word r	Compensation word c	v_c
MSR-4	$0 \parallel b_4 b_3 b_2 b_1$	Not consumed by the CPE	0
Non-MSR-4	$1 \parallel b_7 b_6 b_5 b_4$	$b_7 \parallel b_3 b_2 b_1$	1

For an MSR-4 weight, bits b_4 through b_1 are retained and b_0 is reconstructed as one in the local MAC datapath. The reduced-path value is therefore

$$\hat{q}_{\text{MSR-4}} = 2 \text{ sext}_4(b_4 b_3 b_2 b_1) + 1. \quad (2.5)$$

This expected-value LSB rule enables the five-bit representation without carrying the original LSB through every Reduced Processing Element (RPE). A non-MSR-4 weight retains its high-nibble con-

tribution in the RPE path and carries its omitted signed residual in the Compensation Processing Element (CPE) path. The two partial products are accumulated only when $v_c = 1$.

Motivation

3.1 Why MSR-4 Is the Selected Operating Point

The MSR length determines the trade-off between compression opportunity and the frequency of compensation. Table 3.1 summarizes the measured MSR coverage of four INT8-quantized MNIST models. MSR-3 is nearly universal but preserves fewer redundant bits. MSR-5 through MSR-7 provide more aggressive reduction, but their coverage becomes model dependent; LeNet falls to 88.3% at MSR-5 and 53.4% at MSR-6. MSR-4 provides the selected balance: it covers at least 98.90% of the reported weights while supporting the 5-bit reduced representation in Table 2.1.

Table 3.1: Observed MSR Coverage of INT8 Weight Data

Run criterion	MLP	LeNet	ResNet	AlexNet
MSR-3	99.9%	99.9%	99.9%	99.9%
MSR-4	99.98%	98.90%	99.61%	99.98%
MSR-5	98.0%	88.3%	99.5%	99.7%
MSR-6	78.2%	53.4%	99.1%	97.8%
MSR-7	40.4%	27.3%	85.5%	84.3%

3.2 MSR-4 Exception Density and Compensation Capacity

The low non-MSR-4 density makes a sparse compensation array practical. For a column with $N = 256$ weights and MSR-4 coverage $p_{\text{MSR-4}}$, the expected number of non-MSR-4 weights is

$$E[N_{\text{non}}] = N(1 - p_{\text{MSR-4}}). \quad (3.1)$$

LeNet is the limiting model in the reported data. With $p_{\text{MSR-4}} = 0.9890$,

$$E[N_{\text{non}}] = 256(1 - 0.9890) = 2.816 \approx 2.9. \quad (3.2)$$

Accordingly, the target 256×256 reduced-precision systolic array is paired with a 256×3 compensation array. The latter contains 768 CPE sites, only $768/65,536 \approx 1.17\%$ of the main-array site count. This ratio does not directly represent area or power overhead, but it establishes that the correction resource can remain narrow when the measured MSR-4 distribution is maintained.

The average in (3.2) is not a worst-case bound. The model-preparation flow shall count non-MSR-4 weights for every mapped tile and column before preload. A column with more than three exceptions must be reported as an overflow and scheduled by a separate mapping or compensation pass; it must not silently discard the excess residuals.

3.3 Expected-Value Approximation and Sparse Compensation

Setting the discarded LSB to one intentionally introduces a bounded approximation for an MSR-4 weight:

$$\epsilon_q = \hat{q} - q = 1 - b_0. \quad (3.3)$$

For a dot product consisting of MSR-4 values, this error accumulates as

$$\epsilon_y = \sum_i x_i(1 - b_{0,i}). \quad (3.4)$$

The assumption must consequently be evaluated with an RTL-equivalent model and compared with INT8 inference; MSR coverage alone is not an accuracy result.

Non-MSR-4 values require a separate mechanism because reducing them only to their high nibble discards information that can materially affect sensitive layers. The CPE path retains the sign and lower residual field for these exceptional weights, while the main RPE array processes the common case. Figure 3.1 summarizes the data decomposition, storage, reduced-precision MAC operation, and correction accumulation that motivate this split.

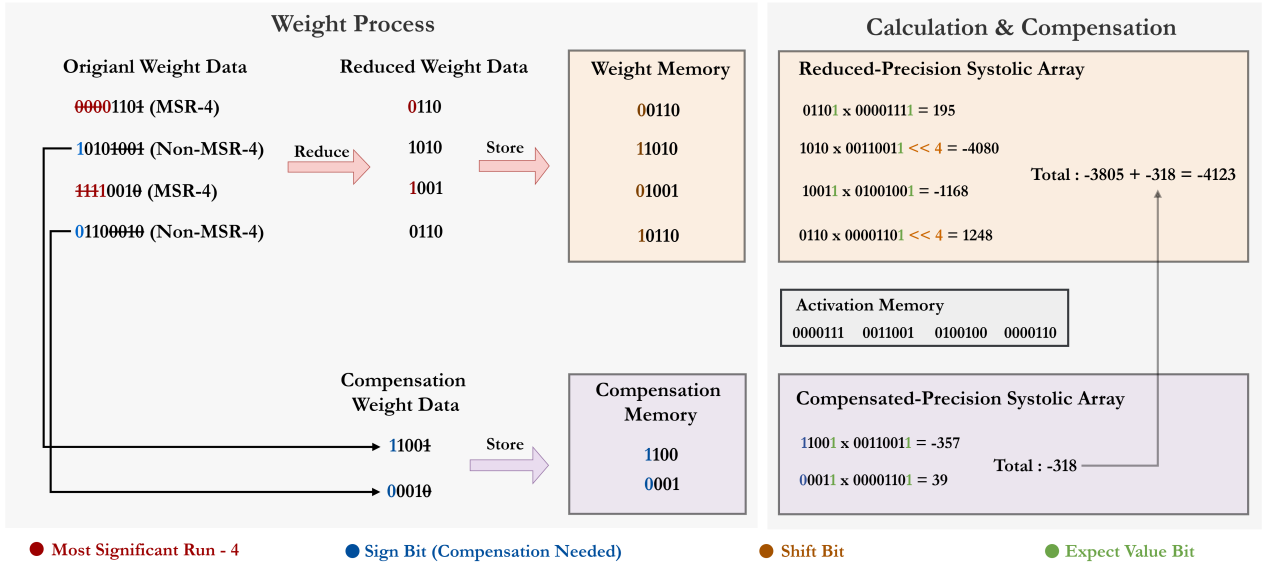


Figure 3.1: MSR-4-aware weight processing and sparse compensation. MSR-4 weights use the reduced path; non-MSR-4 weights produce a reduced contribution and a compensation contribution that are merged in the accumulator.

3.4 Verification Implications

The WPU behavior follows directly from Table 2.1. The following representative vectors shall be observed: 00001101₂ produces $r = 00110_2$ and $v_c = 0$; 11110110₂ produces $r = 01011_2$ and $v_c = 0$; and 01010110₂ produces $r = 10101_2$, $c = 0011_2$, and $v_c = 1$. The WPU verification plan shall exhaustively cover all 256 INT8 input words in addition to these representative vectors.

Bibliography

- [1] “Low-Cost AI Accelerator Based on TPU v1,” project README, tpu-accelerator-lite, accessed July 2026.
- [2] “Weight-Aware and Reduced-Precision Architecture Designs for Low-Cost AI Accelerators,” National Taiwan University of Science and Technology repository. [Online]. Available: <https://hdl.handle.net/11296/zs6qk8>